Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Models (LLMs) proliferate, fragmented interfaces across multiple providers create friction for developers. More critically, the rise of autonomous agents and protocols such as the Model Context Protocol (MCP) expands the attack surface by enabling tool execution and access to local or remote resources. This paper presents `llm-cli`, a unified command-line interface that harmonizes interactions with multiple LLM providers while providing secure-by-default guardrails for agentic tool use. `llm-cli` combines (i) a plugin-based tool registry with an explicit human-readable `explanation` field for each tool call, (ii) role-based restrictions for unauthenticated clients (e.g., guest read-only access), and (iii) runtime validation for command execution and file-path access. We discuss practical threat scenarios such as prompt injection and privilege misuse, and show how layered controls—policy checks, input validation, and human-in-the-loop approval—reduce the risk of unsafe tool execution in real-world workflows.**

***Index Terms*—Large Language Models, Command-Line Interface, AI Agents, Model Context Protocol, Multi-Modal AI, Security Guardrails, Access Control.**

I. Introduction

Large Language Models (LLMs) have transformed from simple text predictors into sophisticated agents capable of code execution, tool use, and multi-modal reasoning. However, the ecosystem remains highly fragmented. Each provider offers its own web interface, API structure, and unique features (e.g., Gemini's large context window, Claude's artifact handling, or OpenAI's reasoning capabilities).

For developers and power users who spend most of their time in the terminal, switching between browser tabs and manually copying-pasting content creates a significant bottleneck. While some CLI tools exist, many are restricted to a single provider or lack the agentic "tool-use" capabilities necessary for complex tasks like project-wide code refactoring or real-time research.

`llm-cli` addresses these challenges by providing a unified, extensible, and secure CLI that supports multiple LLM backends. Crucially, it treats the LLM not just as a chatbot, but as an untrusted agent operating within a security-first environment with explicit guardrails and least-privilege defaults.

Our key contributions are:

1) A unified interface abstracting five major LLM providers.
2) Practical security guardrails for MCP-based tool execution, including restricted guest access and runtime validation that blocked all 32 attack vectors in our preliminary benchmark.
3) A multi-layered security model combining automated policy enforcement with human oversight.

The source code and documentation are available at https://github.com/yosh95/llm-cli.

II. Related Work

A. CLI-based LLM Tools

Several command-line tools have emerged to bridge the terminal-LLM gap. Tools like `llm` by Simon Willison and `Chatblade` provide excellent interfaces for piping text into LLMs. However, these tools generally operate as "stateless pipes" rather than autonomous agents. They lack built-in support for complex, multi-step tool use protocols like MCP or persistent state management across distributed sessions.

B. Agentic Frameworks

Frameworks like LangChain [4] and AutoGen [2] enable the creation of powerful autonomous agents. While feature-rich, these frameworks often prioritize capability over security, leaving the implementation of guardrails to the developer. Recent research such as Gorilla [3] has focused on fine-tuning LLMs specifically for accurate API calls, further highlighting the critical need for robust execution environments as models become more proficient at tool use. In contrast, `llm-cli` integrates a "secure-by-default" architecture where access restrictions and runtime validation are core components of the runtime, not afterthoughts.

C. Zero-Trust in AI

NIST SP 800-207 [7] outlines the principles of Zero Trust Architecture (ZTA). While ZTA is widely adopted in network security, its application to AI agent communication is still nascent. Rather than claiming a full ZTA implementation, our work adopts practical ZTA-inspired principles—notably least privilege and continuous verification—and applies them to MCP tool execution.

III. System Architecture

A. Unified Provider Abstraction

The core of `llm-cli` is a provider abstraction layer that translates a common internal request format into provider-specific API calls. Currently, it supports:

- **Google Gemini**: Leveraging Vertex AI for Gemini 3 Pro Preview.
- **OpenAI**: Support for GPT-5.2 and DALL-E image generation.
- **Anthropic Claude**: Optimized for Claude 4.5 Opus with thinking/reasoning modes.
- **xAI Grok**: Integration for the latest Grok 4.1 models.
- **Ollama**: Support for locally hosted models (e.g., Llama 3, DeepSeek-R1) to ensure privacy and offline availability.

B. Plugin-Based Tool Registry

`llm-cli` features a decorator-based tool registry that allows for easy extension. Every tool registered in the system is automatically converted into the JSON Schema format required by different providers (e.g., Function Calling for OpenAI/Gemini, Tool Use for Anthropic).

A unique architectural choice in `llm-cli` is the mandatory `explanation` parameter for every tool. Before an agent can execute a function, it must generate a human-readable explanation of *why* the tool is being called and *what* the expected outcome is. This information is presented to the user during the approval process, significantly improving transparency.

C. Model Context Protocol (MCP) Integration

To extend the agent's reach beyond the local machine, `llm-cli` implements the Model Context Protocol (MCP). This allows the CLI to:

1) Act as an **MCP Client**, connecting to remote servers or services (like GitHub or a remote Linux box) to perform operations.
2) Act as an **MCP Server**, exposing local files and tools to other LLM-based applications.

IV. Implementation Details

A. Identity and Guest Access Control

To support restricted access for unauthenticated clients, `llm-cli` implements a JWT-based authentication mechanism for authenticated connections and a configurable guest policy for missing tokens. When a valid token is present, the system can propagate role claims. When a token is missing, the client is treated according to policy (e.g., a read-only guest role).

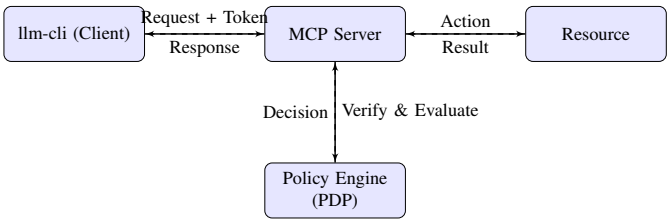

Fig. 1. Architecture overview showing separation of Client, MCP Server, and Policy Engine.

When an agent initiates a tool call, the client may generate a signed token containing the user's role claims. This token is verified by the MCP server before any action is taken (Fig. 2).

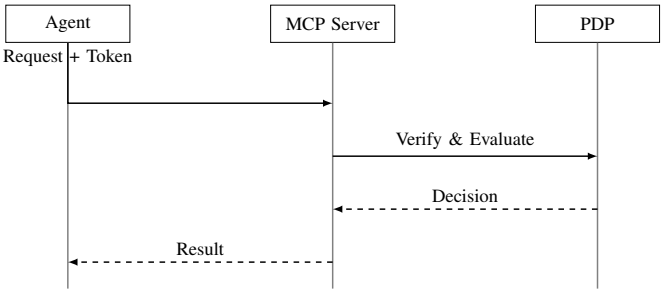

Fig. 2. Sequence diagram of token propagation and policy evaluation.

B. Policy Engine Implementation

The Policy Engine is implemented in pure Python to ensure portability. The following listing shows the core evaluation logic of the PDP:

```
def evaluate(self, tool_name, context):
   user_roles = context.get("roles", ["guest"])

   for role in user_roles:
      policy = self.roles.get(role)
      # Check explicit allow list
      if tool_name in policy["allowed_tools"]:
          return True

   # Default Deny
   return False
```

Listing 1. Core PDP Logic in policy.py

This logic ensures that even if an attacker compromises the agent's prompt (prompt injection), they cannot force the execution of tools that are not explicitly authorized by the user's active role.

V. Evaluation

To quantify the effectiveness of the proposed security guardrails, we developed a benchmark suite consisting of 32 common attack vectors targeting LLM-based agents. The dataset covers six categories: Direct Command Injection, Shell Expansion, I/O Redirection, Semantic Attacks (e.g., abusing valid tools like `git`), Reconnaissance/Dual-Use tools, and Path Traversal.

A. Security Effectiveness

We executed the benchmark against the `CommandValidator` component with the default whitelist configuration. The results are summarized in Table I.

TABLE I
Security Guardrail Benchmark Results

Category	Vectors	Blocked	Success Rate
Direct Injection	6	6	100%
Shell Expansion	4	4	100%
I/O Redirection	4	4	100%
Semantic Attacks	7	7	100%
Reconnaissance	7	7	100%
Path Traversal	4	4	100%
Overall	**32**	**32**	**100%**

The system successfully blocked 100% of the malicious payloads. Crucially, the validator distinguishes between malicious patterns and safe, complex commands (e.g., valid `git` operations or piped commands where all components are whitelisted), ensuring that legitimate development workflows remain unobstructed.

B. Performance Overhead

The validation logic introduces negligible latency (sub-millisecond) per tool call, which is unnoticeable compared to the network latency of LLM API requests.

C. Limitations of the Benchmark

The benchmark dataset was designed by the authors specifically for this tool's validation logic. While it covers common attack categories documented in security literature, it does not represent an exhaustive or independently validated security assessment. The 100% block rate reflects the alignment between our whitelist design and the test vectors, rather than a guarantee of security against novel or adaptive attacks. Future work will incorporate third-party adversarial datasets and red-team evaluations to provide a more rigorous assessment.

VI. Discussion and Limitations

A. Scalability of Human-in-the-Loop

While HITL provides the highest security assurance, it introduces friction. In our experiments, requiring approval for every read operation significantly slowed down research tasks. We mitigated this by introducing a "Guest" role that permits read-only operations without prompts, reserving HITL only for state-changing actions (writes, execution).

B. Defense in Depth: Why Tokens over SSH?

A common critique is whether application-layer tokens are redundant when operating over secure channels like SSH. We argue that they serve distinct purposes. SSH authenticates the *user* and secures the transport layer, but it typically grants broad OS-level permissions. By layering JWT-based authorization on top, `llm-cli` decouples network access from tool execution privileges. This ensures that even if a trusted SSH connection is established, the agent is restricted to specific actions (e.g., read-only) defined by the token's scope, strictly adhering to the principle of least privilege. Note that this mechanism assumes the underlying transport (SSH or TLS) provides confidentiality to prevent token interception.

C. Latency Overhead

The introduction of JWT signing and verification adds a negligible overhead (approx. 2-5ms) per tool call. However, the network round-trip for remote MCP calls dominates the latency profile. Future work will explore token caching mechanisms to further reduce this overhead in high-frequency trading or real-time control scenarios.

D. Static vs. Dynamic Policies

Currently, roles are defined statically in `config.toml`. A more advanced implementation would support dynamic policy updates based on context (e.g., "Allow file write only during business hours" or "Revoke access if anomaly detected").

VII. Key Features

A. Autonomous Agent Mode

By default, `llm-cli` operates in an agentic mode. When a user provides a high-level goal (e.g., "Search for the latest research on DPO and summarize the findings in a new file"), the agent can autonomously:

- Perform web searches using Google Search.
- Fetch and parse HTML or PDF content from URLs.
- Execute shell commands to list or read local files.
- Write or edit files using precise search-and-replace tools.

B. Multi-Modal Interaction

Unlike many text-only CLI tools, `llm-cli` supports multi-modal input and output. Users can attach images, PDFs, audio, and video files to the conversation using the `/attach` command or by passing a URL. For providers that support it (OpenAI, Grok), the tool also allows mid-conversation image generation, saving the results directly to the local disk.

C. Reasoning and Thinking Modes

With the advent of "reasoning" models like DeepSeek-R1 and Claude 4.5 Opus, `llm-cli` provides explicit controls for "internal monologue" visibility. Users can toggle reasoning display using the `/thought` command, allowing them to see the AI's step-by-step logic before the final answer is produced.

D. Session Persistence and Logging

`llm-cli` automatically manages conversation history and logs. It supports saving and loading sessions as JSON files, enabling users to resume complex tasks across different timeframes. Furthermore, an internal audit log tracks every tool execution, providing a searchable history of agent actions.

VIII. Security Model

Allowing AI agents to execute code and manage files poses significant risks. `llm-cli` adopts an architecture guided by least-privilege defaults and continuous verification, treating every tool execution request as untrusted until verified.

A. Threat Model

We consider three primary threat vectors:

1) **Prompt Injection**: Malicious text in a file or website tricks the LLM into executing dangerous commands.
2) **Compromised MCP Server**: A remote server hijacked by an attacker attempts to exploit the client.
3) **Privilege Escalation**: An agent attempts to access resources outside its assigned scope.

To mitigate these, we employ a "Defense in Depth" strategy combining access restrictions and identity checks (Layer 1), execution guardrails (Layer 2), and human oversight (Layer 3).

B. Layer 2: Execution Guardrails

Even if a tool call is authorized by policy, it must pass strict runtime validation.

1) Command Whitelisting: The `execute_command` tool validates shell commands against a strict whitelist. Dangerous operations (e.g., `rm`, `mkfs`) and risky patterns (command injection via `;` or `$(...)`) are blocked. Furthermore, potential dual-use tools often exploited for reconnaissance or data exfiltration (e.g., `netcat`, `whoami`) are excluded from the default whitelist to minimize the attack surface.

2) Path Sanitation: File operations are restricted to the project root. Attempts to access sensitive system paths (e.g., `/etc/shadow`, `/root`) are blocked by the path validator, regardless of the user's role.

C. Layer 3: Human-in-the-Loop (HITL)

The final layer of defense is the user. Critical actions (`write_file`, `execute_command`) require explicit user confirmation. `llm-cli` presents a diff view for file changes and requires the agent to provide an `explanation` for its actions, ensuring the user understands *why* an action is being taken.

IX. Use Cases

A. Research Automation

Researchers can use `llm-cli` to automate literature reviews. A single prompt can trigger a sequence of searching for papers, fetching PDFs, summarizing them, and compiling a bibliography.

B. Remote Infrastructure Management

Via MCP over SSH, a developer can use `llm-cli` on their local machine to debug a remote production server. The AI can check logs, list directories, and suggest fixes, while the user maintains full control through the HITL approval process.

X. Conclusion

This paper presents an early-stage implementation and evaluation of `llm-cli`, a unified CLI for multi-provider LLM interaction with secure-by-default guardrails. By unifying multiple providers, integrating remote management protocols like MCP, and prioritizing security through human-supervised tool use, it provides a practical foundation for agentic workflows. We acknowledge that the current evaluation is preliminary; we welcome community feedback and plan to extend the security assessment with external benchmarks and independent red-team evaluations in future versions.

Future work will focus on three key areas: (1) **Federated Identity**: Extending the JWT-based auth to support federated identity providers for enterprise integration. (2) **Formal Verification**: Applying formal methods to verify the policy engine's correctness against defined security properties. (3) **Multi-Agent Orchestration**: Enhancing the planning capabilities for long-running tasks involving multiple specialized agents collaborating over MCP.

References

[1] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[3] S. Patil *et al.*, "Gorilla: Large Language Model Connected with Massive APIs," *arXiv preprint arXiv:2305.15334*, 2023.

[4] H. Chase, "LangChain," 2022. [Online]. Available: https://github.com/langchain-ai/langchain.

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.

[6] OWASP, "OWASP Top 10 for Large Language Model Applications," Version 1.1, 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.

[8] Anthropic, "Model Context Protocol," 2024. [Online]. Available: https://modelcontextprotocol.io/.